4.3.1 Instrumentation

In the context of the code you own, the first thing you want to tackle is emitting signals, using OpenTelemetry. OpenTelemetry provides instrumentation support for 11 programming languages at the time of writing. More specifically, OpenTelemetry supports C++, .NET, Erlang/Elixir, Go, Java, JavaScript, PHP, Python, Ruby, Rust, and Swift.

In OpenTelemetry, instrumentation is organized such that users get a library software development kit for each programming language and, potentially, support for automatic instrumentation. Some languages offer this via separate repos (e.g., Java), while others, like Ruby, maintain a single repo that supports both manual and auto-instrumentation. Remember that instrumentation doesn't come for free. Even in the best case of auto-instrumentation, where you don't have to change your code, there will be a small impact on your app's performance (http://mng.bz/Rx5j).

Let's have a closer look at the different instrumentation methods in OpenTelemetry. We will start with automatic instrumentation, since it's a great starting point and requires less effort.

Automatic instrumentation

With automatic instrumentation, or simply auto-instrumentation, in OpenTelemetry, the steps required are as follows:

1. First, enable auto-instrumentation. You typically need to add one or more dependencies. At a minimum, these dependencies will add OpenTelemetry API and SDK capabilities.
2. Then, configure the auto-instrumentation, either via environment variables (and/or language-specific mechanisms, such as system properties in Java). You must, at the very least, provide a service name to identify the service you're instrumenting and, optionally, other configuration options, including things such as source configuration, exporter configuration, and resource configuration.

OpenTelemetry (at the time of writing) provides auto-instrumentation for generating traces for certain programming languages, including

- Java auto-instrumentation (https://github.com/open-telemetry/opentelemetry-java-instrumentation), which is part of OpenTelemetry, in the form of a JAR file you include (using `-javaagent`). You can configure this agent in a fine-grained manner (http://mng.bz/2DQ8), from sanitizing database queries to capturing predefined HTTP headers, which will dynamically inject JVM bytecode to capture signals from popular Java frameworks.
- JavaScript (https://github.com/open-telemetry/opentelemetry-js), also officially supported via the OpenTelemetry core, also allows yout to use all of your JavaScript apps. Note there are additional, framework-specific SDKs, such as the OpenTelemetry Node SDK (http://mng.bz/1q41), that enable auto-instrumentation.